

✓ Step 1 — Import Libraries

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

✓ Step 2 — Load Dataset

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
#Upload file
path='/content/gdrive/MyDrive/XYZ.csv'
df = pd.read_csv(path)
```

Preview dataset

```
# First 5 rows
df.head()
```

```
#Check data types and missing values
df.info()
```

✓ Step 3 — Select Features and Labels

```
# Feature and Target
X = df[["Feature1", "Feature2"]]
y = df["Label"]
```

✓ Step 4 — Check Class Distribution

Interpretation:

First value → class 0 count

Second value → class 1 count

```
#checks whether the dataset is balanced
print(y.value_counts())
```

```
#Another way
np.bincount(y)
```

✓ Step 5 — Feature Distribution Analysis

Boxplot shows:

Median

Quartiles

Outliers

Class separation

```
#Boxplot
sns.boxplot(x=y, y=X["Feature 1"])
plt.show()
```

✓ Step 6 — Train-Test Split

Train dataset- 70% Test dataset- 30%

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

✓ Step 7 — Data Scaling (Standardization)

```
#Data normalization
scaler = StandardScaler()

# fit on training data
X_train = scaler.fit_transform(X_train)
# fit on test data
X_test = scaler.transform(X_test)
```

✓ Step 8 — Model Training

class_weight='balanced' handles class imbalance.

```
#Model selection and training
model = LogisticRegression(class_weight='balanced')
model.fit(X_train, y_train)
```

✓ Step 9 — Prediction

```
#Prediction
y_pred = model.predict(X_test)
```

✓ Step 10 — Model Evaluation

```
#Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

✓ Step 11 — Model Coefficients

```
print("\nModel Coefficients:")
for name, coef in zip(X.columns, model.coef_[0]):
```

```
print(f"{name}: {coef:.4f}")
```

```
intercept = model.intercept_  
intercept
```